COMPUTER NETWORKS | EXAM PREPARATION

From first principles to 5-mark answers

Lab Q6, Q8, Q10-Q14 + every topic in your screenshot

How to use this guide

Start with the two foundation pages. Read **Understand it first**, follow the diagram, then learn the separate **5-mark answer**. The explanations assume that you have not studied Units 3-5.

Part A explains the seven lab questions and connects the theory to your C programs. Q14 is a command exercise. Part B covers the screenshot topics as descriptive semester answers; "viva" does not limit their depth.

For a typical 5-mark response, use a definition, a clearly labelled diagram where applicable, and 4-6 relevant points. Add the short worked example when the question calls for an algorithm. This is an answer-writing structure, not a promise about your examiner's mark allocation.

Read in this order	Topics
Foundations: pp.2-3	Networks, routing, IP/MAC/ports, headers and layers
Lab theory: pp.4-10	Classful; Dijkstra; distance vector; link state; leaky bucket; encryption; troubleshooting
Unit 3: pp.11-17	IPv4, IPv6, ICMP, ARP, RARP, DHCP, link state
Unit 4: pp.18-21	TCP services, connection management, TCP header, UDP header
Unit 5: pp.22-29	DNS, email agents, FTP, TFTP, Telnet, POP3, SMTP, IMAP
Revision/sources: pp.30-36	Comparison tables, recall checks, source diagrams and references

Diagram policy

Teaching diagrams are simplified redrawings based on the cited notes/textbook/protocol descriptions. Exact crops from your Tanenbaum PDF are also included in the source-diagram appendix. Header widths have been checked. Generated numerical examples are labelled as practice.

Your confirmed correction: cover **IMAP** under email. **ICMP** is explained separately as an IP support protocol. The crossed-out duplicate "Email" entry is not treated as a separate question.

Prepared for revision on 10 November 2026. This guide covers the requested questions, not every topic in Units 3-5.

START HERE

What is a network, and what is routing?

Understand it first

A computer network connects devices so that they can exchange data. A **host** is an end device, such as your laptop or a server. A **link** is a connection carrying data. A **router** connects different IP networks and sends packets toward their destinations.

Imagine sending a parcel from Hyderabad to Delhi. Your house address identifies where it starts, the delivery address identifies where it must finish, and intermediate sorting centres decide where to send it next. In a network, an **IP address** identifies a network interface, a **packet** carries data and addressing information, and routers act like sorting centres.

Routing means determining which paths or next hops should be used. **Forwarding** means actually sending an arriving packet out through the selected interface. A routing algorithm builds the information; a router uses that information to forward packets.

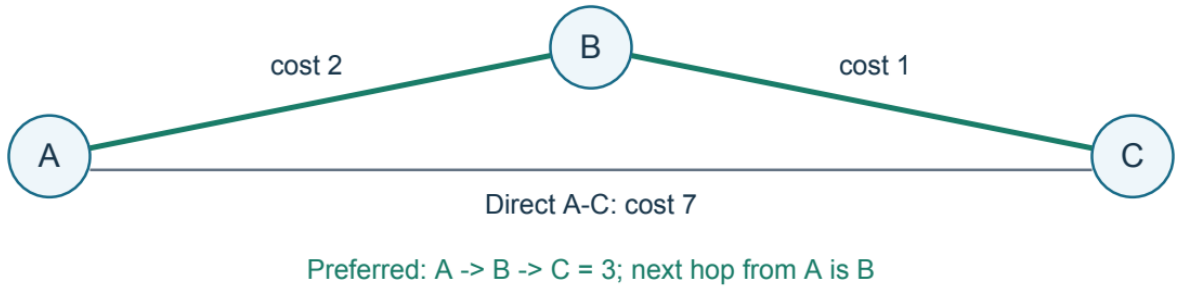


Diagram to draw: Draw the three-router graph. Write each link cost beside its line. Mark A → B → C as the preferred route.

The words you need

1. A **route** is a path through the network. A **hop** is one step from a router to the next device/router.
2. A **neighbour** is directly connected. A **next hop** is the first neighbour used toward a destination; it need not be the final destination.
3. A **routing table** records a destination, a next hop/outgoing interface, and often a metric or cost. Real IP routing usually uses destination network prefixes; the lab represents destinations as router numbers.
4. A **metric** measures how desirable a route is. It may represent hop count, delay, or a configured cost. Add the link costs to obtain the path cost.

Worked example / explanation: Practice network: A-B costs 2, B-C costs 1, and A-C costs 7. Direct A → C costs 7. A → B → C costs 2 + 1 = 3. So A sends a packet for C first to B. The next hop is B; the destination remains C.

Remember: Destination = where it must finish. Next hop = where it goes immediately.

Source: U3 pp.2-3; Tanenbaum ch.5, routing algorithms.

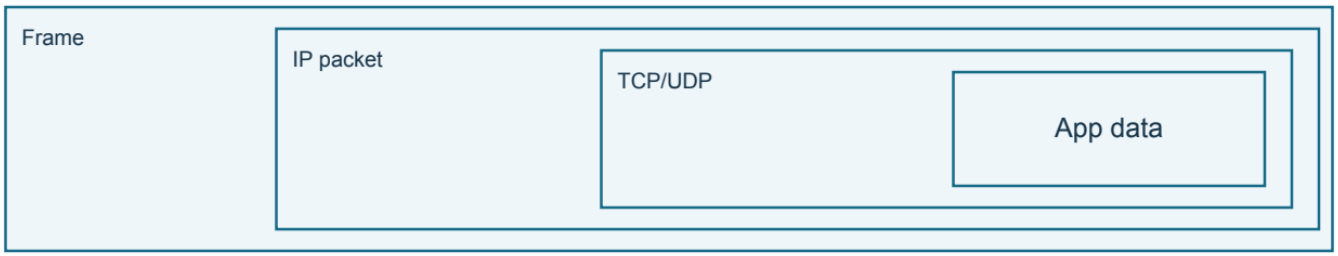
START HERE

IP address, MAC address, port and header

Understand it first

These four words answer different questions. **IP** identifies the network interface to reach. A **MAC address** identifies a link-layer interface for delivery on a local link. A **port number** identifies a transport endpoint used by an application. A **header** is control information placed before the actual data.

For a remote website, your laptop normally sends the first Ethernet/Wi-Fi frame toward its default gateway. The IP destination is the remote server, but the first local frame is addressed to the gateway. This is why ARP asks for the gateway's MAC, not the remote server's MAC.



Headers are added as data moves down the lavers.

Diagram to draw: Draw nested boxes: frame header | IP header | TCP/UDP header | application data. Label the layers.

The words you need

1. **Application layer:** provides named services such as DNS, email, FTP and Telnet.
2. **Transport layer:** TCP/UDP deliver data between application endpoints using ports. TCP adds reliability and ordering; UDP provides a small datagram service.
3. **Network layer:** IP provides addressing and packet forwarding across networks. Routing chooses the path. ICMP reports network errors and supports diagnosis.
4. **Data-link layer:** carries frames across a local link, using addresses such as Ethernet MAC addresses.
5. **Encapsulation:** application data becomes transport payload, the transport segment/datagram becomes IP payload, and the IP packet becomes frame payload. The receiver removes the headers in reverse order.

Worked example / explanation: A 32-bit IPv4 address contains 4 groups of 8 bits. Each group is an octet (one byte), so each decimal group ranges from 0 to 255. Header diagrams normally have 32 bits across one row: that row is 4 bytes.

Remember: 1 byte = 8 bits. IP reaches a host/interface; a port reaches the application endpoint.

Source: Tanenbaum chs.1, 5 and 6; U3 pp.57-64; U4 p.6.

PART A - LAB THEORY

Q6. Classful addressing

Understand it first

An IPv4 address has a **network part** and a **host part**. The network part identifies the network; the host part distinguishes interfaces within that network. A **subnet mask** marks network bits with 1s and host bits with 0s.

Classful addressing is a historical way to choose this split from the first few bits of the address. For the lab, inspect the first decimal octet. Class A uses the first octet as the network part, Class B uses two, and Class C uses three.

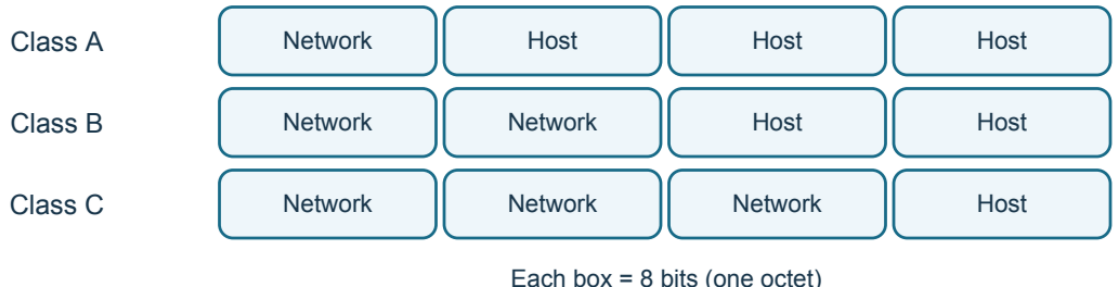


Diagram to draw: Draw the three address rows: A = N|H|H|H; B = N|N|H|H; C = N|N|N|H. Each box is one octet.

5-mark answer

1. IPv4 uses 32-bit addresses written as four decimal octets, such as 192.168.1.10.
2. Classes A, B and C provide different network/host divisions. Their default masks are /8, /16 and /24 respectively.
3. Class D addresses are used for multicast groups. Class E is reserved; D and E do not use the ordinary A/B/C network-host division.
4. The network address is obtained by applying the mask (bitwise AND). With a default Class C mask, set the final octet to zero.
5. For ordinary A/B/C networks, usable host count is traditionally 2^h - 2, excluding the all-zero host part and all-one host part. Modern networks use classless prefixes; the lab specifically asks for classful classification.

Class	First octet	Default mask	Host bits
A	1-126*	255.0.0.0 (/8)	24
B	128-191	255.255.0.0 (/16)	16
C	192-223	255.255.255.0 (/24)	8
D	224-239	Multicast	-
E	240-255	Reserved	-

Worked example / explanation: 192.168.1.10 is Class C. Mask = 255.255.255.0; network = 192.168.1.0; host part = 10. *The Class A leading-bit range is 0-127, but 0 and 127 are special ranges; 127 is loopback.

Remember: Program link: read a.b.c.d → compare a with class boundaries → print class, mask and network/host parts.

Source: U3 pp.50-56; Tanenbaum printed pp.449-450 (PDF pp.473-474).

PART A - LAB THEORY

Q8. Dijkstra's shortest-path algorithm

Understand it first

A network can be drawn as a **weighted graph**: vertices represent routers, edges represent links, and weights represent costs. Dijkstra finds the lowest total cost from one source to every reachable destination.

A tentative distance is the best route found so far. A finalized distance is known to be shortest. With nonnegative link costs, the nearest unvisited node can safely be finalized. **Relaxation** means replacing a distance when a better route is discovered.

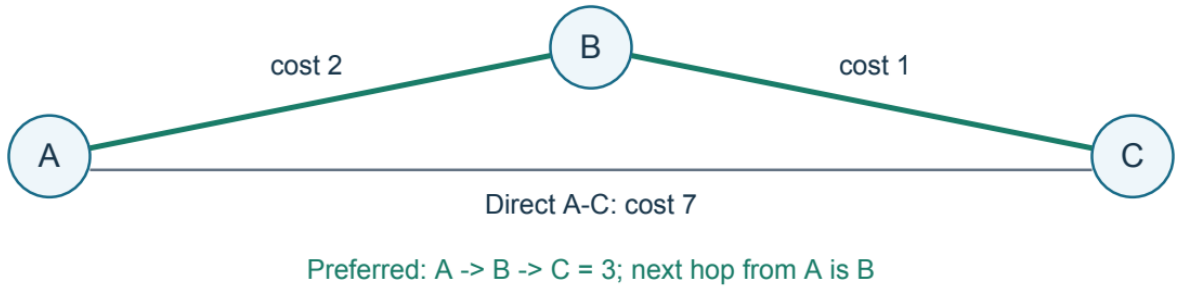


Diagram to draw: Draw A-B=2, B-C=1, A-C=7, and show the distance table below. This is a generated practice example.

5-mark answer

1. Initialize the source distance to 0 and every other distance to infinity. Mark every node unvisited.
2. Choose the unvisited node u with the smallest tentative distance and mark it visited.
3. For each unvisited neighbour v, compare the old distance with dist[u] + cost[u][v]. If the new value is smaller, update the distance and predecessor.
4. Repeat until all reachable nodes are finalized. Stop if no reachable unvisited node remains.
5. Use predecessor links to reconstruct the shortest path. The simple matrix implementation takes O(n^2) time for one source and requires nonnegative weights.

Step	Finalized	Distance A	Distance B	Distance C
Initialize	None	0	Infinity	Infinity
Visit A	A	0	2 via A	7 via A
Visit B	A, B	0	2	3 via B
Visit C	A, B, C	0	2	3

Worked example / explanation: At B: current cost to C is 7; going via B costs 2 + 1 = 3. Replace 7 by 3 and store B as C's predecessor. Path to C: A → B → C.

Remember: Program link: dist = best cost; visited = finalized; parent = previous node. Initialize → minimum → visit → relax.

Source: U3 pp.2-3; Tanenbaum section 5.2.2.

PART A - LAB THEORY

Q10. Distance vector routing

Understand it first

A **distance vector** is a list of a router's best-known distances to destinations. Instead of learning the whole network map, a router asks its direct neighbours what distances they know.

Imagine A asking B, "How far are you from C?" B says 1. A already knows reaching B costs 2, so it calculates a route to C costing 2 + 1 = 3. If that is better than its current cost 7, A changes its next hop for C to B.

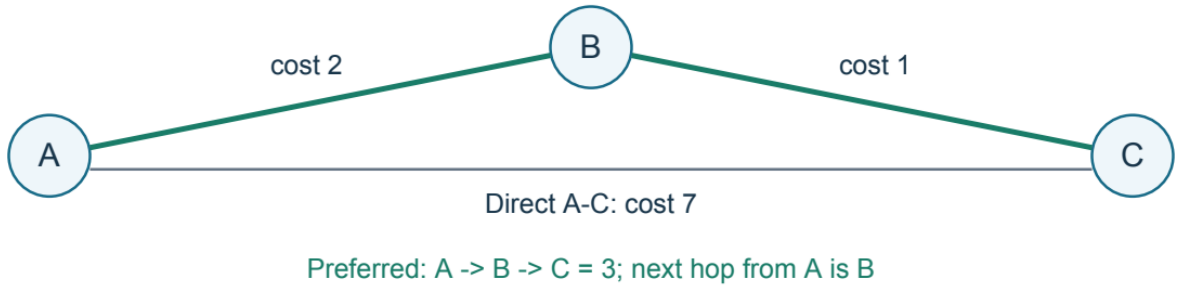


Diagram to draw: Draw the three-router network, then A's routing table showing destination C, next hop B and cost 3.

5-mark answer

1. Each router stores a table containing destination, minimum known distance and next hop.
2. Initially, its own distance is 0, direct neighbours have their link costs, and unknown destinations have infinity.
3. Routers exchange distance vectors with their direct neighbours. For each destination y, router x considers each neighbour v.
4. The Bellman-Ford rule is D_x(y) = min_v [c(x,v) + D_v(y)]. Here c(x,v) is the DIRECT cost to neighbour v, and D_v(y) is v's advertised distance.
5. When a cheaper route is found, update the distance and next hop. Repeat updates until tables stop changing (convergence).
6. It is simple and distributed, but changes can spread slowly and loops/count-to-infinity can occur after failures. RIP is a familiar distance-vector example.

At router A	Direct cost	Via B	Chosen next hop
To C	7	2 + 1 = 3	B

Worked example / explanation: Count-to-infinity: after a route fails, neighbours may incorrectly believe each other still has a working path. They repeatedly increase the advertised distance. The simple lab handles a fixed graph, not this changing-network failure scenario.

Remember: Program link: i = current router, j = destination, k = neighbour. Use cost[i][k] + dist[k][j] for both the comparison and update.

Source: Tanenbaum section 5.2.4, printed pp.370-373 (PDF pp.394-397).

PART A - LAB THEORY

Q11. Link state routing

Understand it first

A **link state** describes a router's direct links: who its neighbours are, what those links cost, and whether they are available. Each router shares this local information so that all routers can build a network map.

This differs from distance vector: "I know my neighbours and their link costs" is shared widely, rather than "Here are my estimated distances to every destination" being shared only with neighbours. Once the map is known, each router runs Dijkstra from itself.

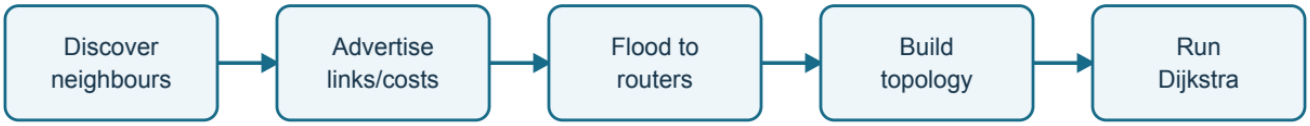


Diagram to draw: For a short exam drawing, use the three-router graph and write: local link information → flooding → topology database → Dijkstra → routing table. The source figure also shows real link-state packet fields.

5-mark answer

1. Discover directly connected neighbours, commonly using HELLO messages.
2. Determine the cost of each direct link.
3. Create a link-state packet containing router identity, neighbour/cost list, sequence number and age.
4. Flood new link-state packets through the network. Sequence numbers reject duplicates/older information, and age removes stale records.
5. Build a common link-state database representing the topology, run Dijkstra, and install the resulting next hops in the routing table.
6. Link state gives a detailed topology view and typically responds to changes more quickly than basic distance vector, at the expense of database storage, computation and flooding overhead. OSPF is a common example.

Worked example / explanation: For the practice graph, A advertises B:2 and C:7. B advertises A:2 and C:1. C advertises A:7 and B:1. Once A has all three descriptions, Dijkstra selects A → B → C, cost 3.

Remember: Program link: the input matrix is the already-known map. The code runs Dijkstra for every source; it does not simulate HELLO messages or flooding.

Source: Tanenbaum section 5.2.5, Fig.5-12; printed pp.373-378 (PDF pp.397-402).

PART A - LAB THEORY

Q12. Leaky bucket and congestion

Understand it first

Congestion occurs when offered traffic exceeds what links/routers can handle: queues grow, delay rises and finite buffers may overflow. **Traffic shaping** controls the pattern of traffic entering the network so sudden bursts become smoother.

Think of water poured irregularly into a bucket with a small outlet. Water can wait inside, but the outlet releases it at a controlled rate. If the bucket cannot hold an arrival, that arrival may be discarded. The lab uses a finite data buffer and a fixed output allowance per time slot.

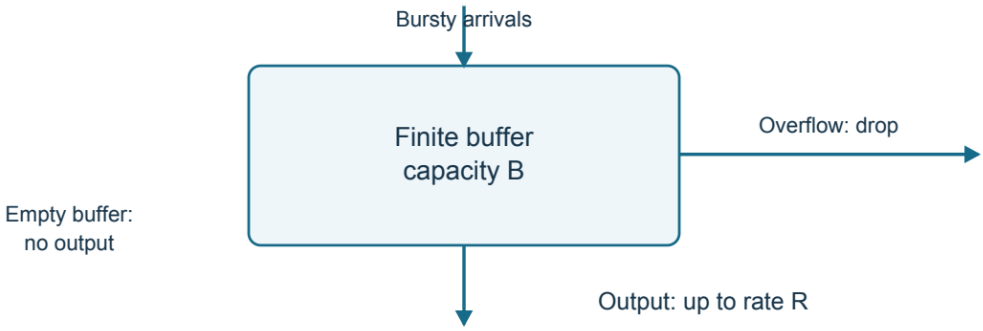


Diagram to draw: Draw a bucket/buffer, a bursty arrival arrow, an overflow/drop arrow, and a bottom arrow labelled fixed rate R. In the book figure, part (b) is the leaky bucket.

5-mark answer

1. Maintain a buffer of capacity B and an output/leak rate R.
2. On each time slot, accept the arriving packet if enough space is available; otherwise drop it in this lab model.
3. Transmit up to R data units: sent = min(current contents, R). Subtract the transmitted amount from the contents.
4. Continue draining after arrivals stop. When empty, send nothing.
5. The method smooths bursts and limits the output rate, helping reduce congestion. Its limitation is that excess traffic can be dropped and queued traffic experiences delay. It does not adapt the leak rate to every network condition.

Time	Arrives	Dropped	Sent	Remaining
1	6	0	3	3
2	8	8	3	0
3	4	0	3	1
4	7	0	3	5

Worked example / explanation: Capacity 10, rate 3: at time 2, 3 units are already stored. An 8-unit packet would require 11 units, so the whole packet is dropped. The stored 3 units are then transmitted. This is the same admission rule used in your program.

Remember: Congestion control protects the network; flow control protects a receiver from being overwhelmed.

Source: U3 p.25; Tanenbaum Fig.5-28, printed p.408 (PDF p.432).

PART A - LAB THEORY

Q13. Encryption and decryption

Understand it first

Plaintext is the original readable message. **Encryption** transforms it into **ciphertext** using an algorithm and a key. **Decryption** uses the appropriate key to recover the original message. The main aim here is confidentiality: preventing an unintended reader from understanding the message.

Your short program uses a Caesar cipher, a substitution cipher that shifts every uppercase letter by the same amount. Both sides use the same shift key, so it is a simple symmetric-key example.

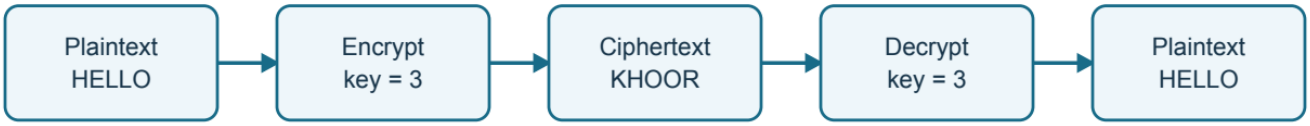


Diagram to draw: Draw plaintext → encryption (key k) → ciphertext → decryption (key k) → plaintext.

5-mark answer

1. Assign numbers A=0, B=1, ..., Z=25. Let the shift key be k.
2. Encryption is C = (P + k) mod 26. Modulo 26 wraps around from Z to A.
3. Decryption is P = (C - k + 26) mod 26. Adding 26 avoids a negative value for the allowed keys.
4. For key 3, HELLO becomes KHOOR. Decrypting KHOOR with key 3 gives HELLO.
5. Symmetric encryption uses a shared secret key; asymmetric encryption uses a related public/private key pair. Caesar is only a teaching example: the small number of shifts makes it easy to break.
6. Encryption alone does not automatically provide integrity or prove who sent a message; those require suitable additional mechanisms.

Worked example / explanation: X corresponds to 23. With k=3: (23+3) mod 26 = 0, so X becomes A. For decryption: (0-3+26) mod 26 = 23, giving X again.

Remember: Program link: subtract 'A' to get 0-25, shift, apply %26, then add 'A'. This version accepts uppercase letters without spaces.

Source: Tanenbaum Fig.8-2 and section 8.1.2; printed pp.767,769 (PDF pp.791,793).

PART A - LAB THEORY

Q14. Network troubleshooting commands

Understand it first

Troubleshooting means locating which stage of communication is failing. A computer may have a bad IP configuration, an unreachable gateway, a DNS problem or a failure farther along the route. Different commands examine different stages.

The syllabus commands are Windows Command Prompt commands. Their output depends on the actual machine and network; memorise the purpose and interpretation rather than a particular printed IP address.

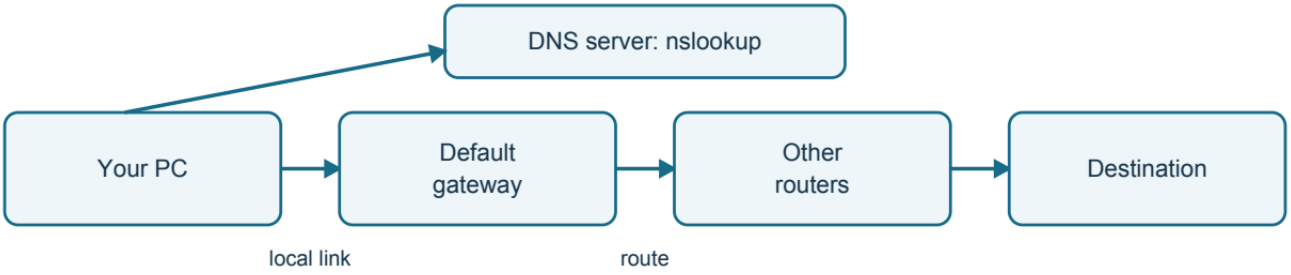


Diagram to draw: Draw PC → default gateway → other routers → destination. Add a separate PC ↔ DNS server branch. Label ping/tracert on the route and nslookup on the DNS branch.

5-mark answer

1. **ipconfig:** displays local IP configuration such as IPv4 address, subnet mask and default gateway. /all adds details including DNS servers and adapter information.
2. **ping:** uses ICMP Echo Request/Reply to test whether a target responds and to measure round-trip time/loss. A timeout alone does not prove that the target is down.
3. **tracert:** sends probes with increasing TTL values and uses replies to reveal responding intermediate hops. A star means the expected reply timed out.
4. **nslookup:** queries DNS, helping determine whether a name resolves and which DNS server answered.
5. **arp -a:** displays cached local IPv4-to-MAC mappings. It is not a list of every host on the Internet.

Command	Useful observation
ipconfig /all	IP, mask, gateway and DNS configuration
ping 127.0.0.1	Local IP stack response; not proof of Internet access
ping <actual gateway>	Whether the local gateway responds
tracert mgit.ac.in	Responding hops toward the destination
nslookup mgit.ac.in	DNS answer or lookup failure
arp -a	Local cached IP/MAC mappings

Remember: Six programs plus one command exercise = the seven lab questions. Do not write a C program for Q14.

Source: Sir's lab Q14; U3 pp.60-64; Microsoft command references in the code guide.

PART B - UNIT 3

IPv4: addressing and datagram header

Understand it first

An IP packet is like an addressed envelope. Its **header** tells the network how to handle it; its **payload** is the carried data. IPv4 provides connectionless, best-effort delivery: it does not itself guarantee arrival, order or recovery from loss.

For a 5-mark "Explain IPv4" question, state address size and service, draw the header, and explain the main field groups. The header has five mandatory 32-bit rows: 5 x 4 = 20 bytes. Options can extend it to 60 bytes.

32 bits across each ordinary row

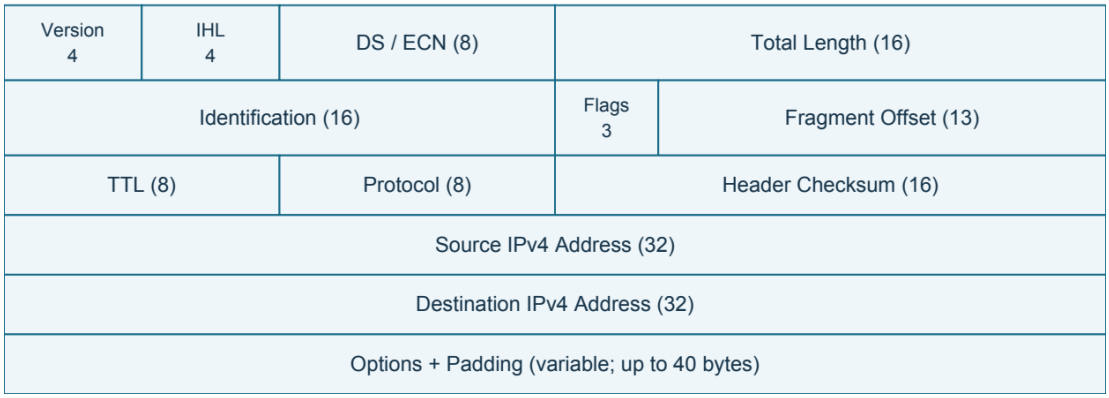


Diagram to draw: Copy the Tanenbaum header. Keep each row 32 bits wide; label the widths using the table. The unlabeled first flag bit is reserved.

5-mark answer

1. IPv4 uses 32-bit addresses, written in dotted-decimal form, and carries each datagram independently through routers.
2. **Version and IHL:** version is 4; IHL gives header length in 32-bit words (5 means 20 bytes). **DS field** carries service/congestion indications.
3. **Total Length** counts the entire IPv4 datagram, header plus payload.
4. **Identification, Flags and Fragment Offset** support fragmentation and reassembly. DF means do not fragment; MF means more fragments follow.
5. **TTL** limits lifetime by decreasing at routers. **Protocol** identifies the carried upper protocol, such as TCP, UDP or ICMP.
6. **Header checksum** detects header corruption. Source and destination addresses identify the endpoints; options/padding are optional.

Fields in order	Widths (bits)
Version | IHL | DS | Total Length	4 | 4 | 8 | 16
Identification | Flags | Fragment Offset	16 | 3 | 13
TTL | Protocol | Header Checksum	8 | 8 | 16
Source address; destination address	32 each

Remember: IPv4 header checksum covers the header, not the payload. IPv4 total length includes the header.

Source: U3 pp.36-38; Tanenbaum Fig.5-46, printed p.439 (PDF p.463).

PART B - UNIT 3

IPv6: features and fixed header

Understand it first

IPv6 expands addressing from 32 bits to **128 bits**. An address is normally written as eight groups of hexadecimal digits separated by colons. Leading zeros within a group may be omitted; one run of zero groups can be compressed using ::.

IPv6 also simplifies the mandatory header. It is always **40 bytes**; optional information is placed in extension headers. Do not copy the outdated 4-bit Priority and 24-bit Flow Label layout in sir's notes.

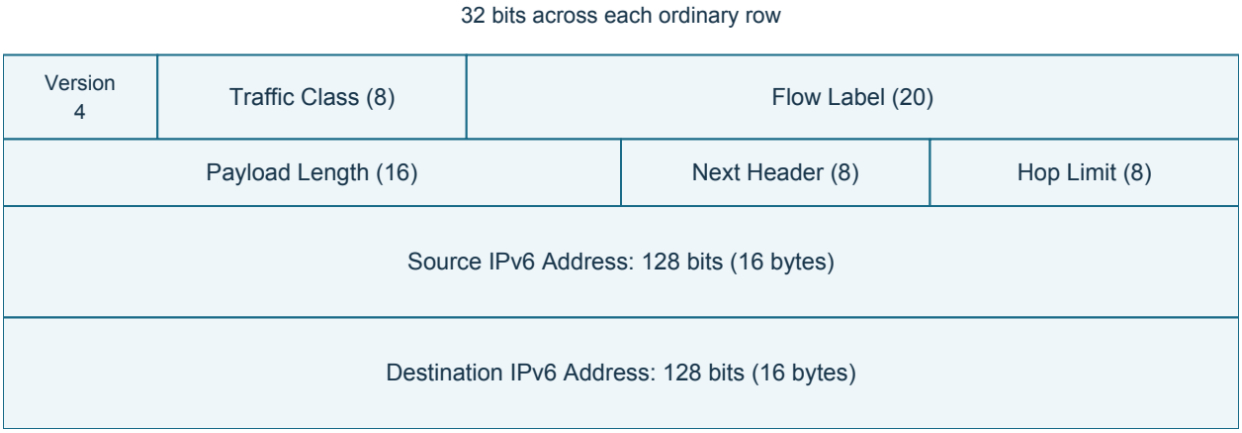


Diagram to draw: Copy the fixed-header diagram. First row: 4 + 8 + 20 bits. Second row: 16 + 8 + 8 bits. Each address occupies four 32-bit rows.

5-mark answer

1. IPv6 provides connectionless IP delivery using a much larger 128-bit address space. Example: 2001:db8::1 is a documentation address.
2. **Version (4 bits)** is 6. **Traffic Class (8)** carries service/congestion information. **Flow Label (20)** labels related packets in a flow.
3. **Payload Length (16)** counts bytes following the fixed header, including any extension headers; it excludes the 40-byte base header.
4. **Next Header (8)** identifies the following extension header or upper-layer protocol. **Hop Limit (8)** is decremented at routers.
5. **Source and Destination Address** are 128 bits each. There is no header checksum in the IPv6 base header.
6. IPv6 uses extension headers for optional functions. Routers do not fragment IPv6 packets; a sending host can use a Fragment extension header. IPv6 uses multicast rather than broadcast.

Worked example / explanation: Base-header size: first two rows = 8 bytes; two 128-bit addresses = 16 + 16 bytes; total = 40 bytes.

Remember: IPv4: 32-bit addresses, 20-60-byte header. IPv6: 128-bit addresses, 40-byte fixed header. IPv6 is not automatically encrypted.

Source: Tanenbaum Fig.5-56, printed pp.458-460 (PDF pp.482-484); RFC 8200, section 3.

PART B - UNIT 3

ICMP: error reporting and diagnostics

Understand it first

IP tries to deliver packets but cannot guarantee success. ICMP gives routers and hosts a way to report certain failures to the sender and to answer diagnostic requests. It reports problems; it does not retransmit lost application data.

For example, if a packet's TTL expires at a router, the router can send an ICMP Time Exceeded message back to the source. This is how traceroute-style tools discover intermediate hops.

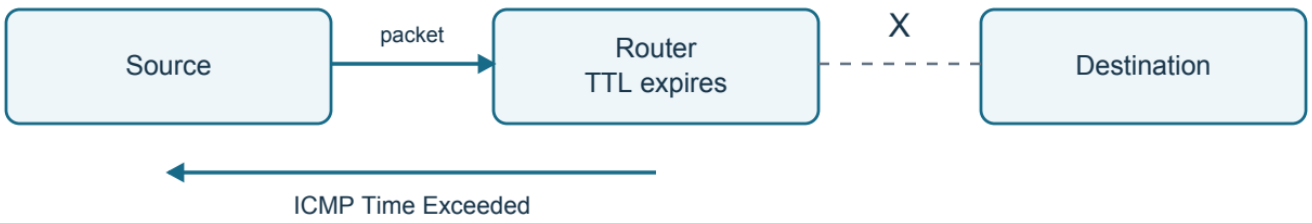


Diagram to draw: Draw Source → Router → Destination; show the packet expiring at the router and a return arrow labelled ICMP Time Exceeded.

5-mark answer

1. Internet Control Message Protocol (ICMP) supports error reporting and diagnostic control for IP. ICMPv4 messages are carried inside IPv4, rather than inside TCP or UDP.
2. The general ICMP header begins with **Type, Code and Checksum**. Type identifies the message category; Code specifies a more detailed reason.
3. **Destination Unreachable** reports that delivery cannot be completed for a stated reason. **Time Exceeded** reports expiry of TTL or a reassembly timer.
4. **Echo Request and Echo Reply** are used by ping to test responsiveness.
5. An ICMP error normally includes information from the triggering packet so the sender can identify it. ICMP does not guarantee delivery of the error message itself.
6. ICMP belongs to the network-layer/IP support family. It is not an email protocol, even if it appears alongside application topics in the course list.

Remember: Ping = Echo. Tracert = increasing TTL and Time Exceeded replies. IMAP, not ICMP, accesses email.

Source: U3 pp.61,63; Tanenbaum section 5.6.4, printed pp.465-466 (PDF pp.489-490).

PART B - UNIT 3

ARP: IPv4 address to MAC address

Understand it first

On an Ethernet LAN, a frame needs a destination MAC address. But an application normally knows an IP address. ARP bridges this gap by finding the MAC associated with an IPv4 next-hop address on the local network.

If the destination is on another network, the next hop is usually a router. In that case ARP resolves the router's local IP address; it does not find the remote server's MAC across the Internet.

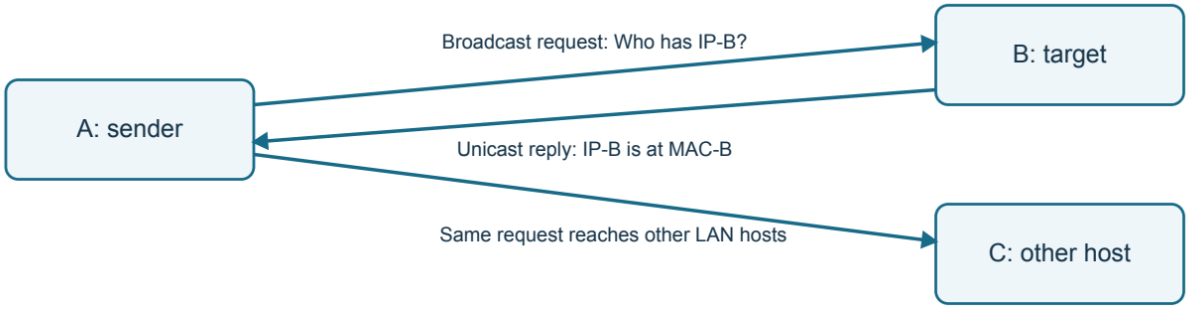


Diagram to draw: Draw sender A, receiver B and another LAN host. Broadcast request to all; unicast reply from B to A. Label the requested IP and returned MAC.

5-mark answer

1. Address Resolution Protocol (ARP) maps an IPv4 address to a link-layer/MAC address on the local network.
2. The sender first checks its ARP cache for an existing IP-to-MAC mapping.
3. If missing, it broadcasts an ARP Request: "Who has this IP address?" All hosts in the local broadcast domain can receive it.
4. The host owning that IP address normally returns a unicast ARP Reply containing its MAC address.
5. The sender caches the mapping and uses the MAC as the destination of the local frame.
6. ARP operates on the local link; routers do not forward its broadcast across IP networks. ARP is carried directly in link-layer frames, not in a TCP or UDP connection.

Worked example / explanation: A knows B's IP 192.168.1.20. A broadcasts "Who has 192.168.1.20?" B replies "192.168.1.20 is at MAC-B". A can now address its local frame to MAC-B.

Remember: DNS: name → IP. ARP: local next-hop IPv4 → MAC.

Source: U3 pp.59-60,64; Tanenbaum printed pp.467-469 (PDF pp.491-493).

PART B - UNIT 3

RARP: MAC address to assigned IP address

Understand it first

A historical diskless workstation could know its own hardware/MAC address at startup but not its IP address. RARP allowed it to ask a local server which IP address had been assigned to that hardware address.

It is not a general Internet search service for discovering arbitrary devices' IP addresses. Its purpose was boot-time address assignment from an administrator-maintained mapping.

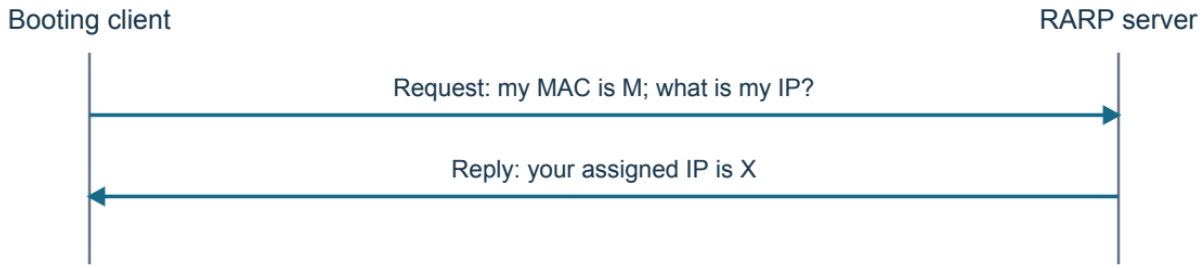


Diagram to draw: Draw client → RARP server: "My MAC is M; what is my IP?" Draw the reply with the assigned IP. Show the MAC/IP table at the server.

5-mark answer

1. Reverse Address Resolution Protocol (RARP) obtains an assigned IPv4 address using a known MAC address.
2. A booting client broadcasts a RARP Request containing its hardware address on the local LAN.
3. A RARP server checks its configured table of MAC-to-IP mappings.
4. The server returns a RARP Reply containing the IP address; the client can then configure its network-layer address.
5. RARP provides only limited configuration and operates on the local link. It does not itself supply the richer gateway, DNS and boot configuration expected from later mechanisms.
6. BOOTP and then DHCP superseded RARP for host configuration. ARP asks for a MAC when the IP is known; RARP asks for an assigned IP when the MAC is known.

Remember: ARP = IP to MAC. RARP = known own MAC to assigned IP. DHCP provides more configuration using leases.

Source: U3 pp.65-66; Tanenbaum printed p.470 (PDF p.494).

PART B - UNIT 3

DHCP: automatic network configuration

Understand it first

A device joining a network needs an IP address, subnet mask, gateway and often DNS server information. Entering these manually on every device is inconvenient. A DHCP server can provide them automatically.

A **lease** is permission to use an address for a limited time. The client can renew it. DHCP is an application-layer protocol carried over UDP, although it configures IP-layer settings.

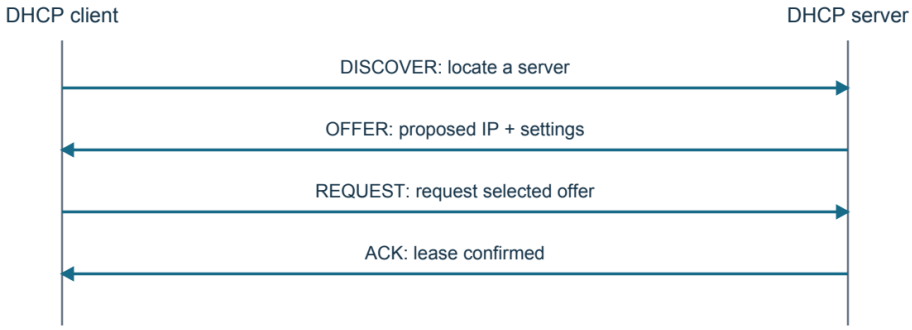


Diagram to draw: Draw two vertical lines, Client and DHCP Server, and four arrows: Discover ->, Offer <-, Request ->, ACK <-. Label the lease/configuration in the final reply.

5-mark answer

1. Dynamic Host Configuration Protocol (DHCP) assigns network configuration to clients, commonly by leasing addresses from a managed pool.
2. **Discover:** an unconfigured client broadcasts to locate DHCP servers.
3. **Offer:** a server offers an address and configuration parameters.
4. **Request:** the client indicates which offered configuration it requests, normally broadcasting this during initial selection.
5. **Acknowledgement:** the chosen server confirms the lease. The client configures its address, mask, gateway and DNS settings.
6. The initial exchange is remembered as **DORA**. DHCPv4 uses UDP server port 67 and client port 68. A relay agent can carry DHCP messages between subnets; ordinary routers do not simply forward the client broadcast.

Remember: DHCP gives configuration; DNS resolves names; ARP discovers a local MAC. They solve different problems.

Source: U3 p.67; U5 p.41; Tanenbaum printed pp.469-470 (PDF pp.493-494).

PART B - UNIT 3

Link state routing: the 5-mark answer

Understand it first

Use the understanding from lab Q11. In a semester answer, explain how the map is obtained as well as how Dijkstra uses it. Writing only Dijkstra's steps omits the link-state advertisement and flooding stages.

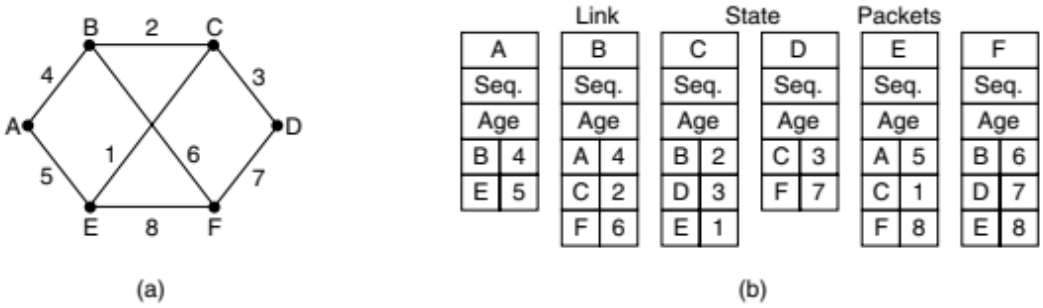


Figure 5-12. (a) A network. (b) The link state packets for this network.

Diagram to draw: Draw the network and the workflow: discover → advertise → flood → build map → Dijkstra → routing table. The textbook figure shows the advertisement contents.

5-mark answer

1. **Definition:** link state routing is a routing method in which routers distribute information about their directly connected links, construct a topology database, and compute shortest paths.
2. **Neighbour discovery:** use HELLO exchanges to learn adjacent routers and determine each link cost.
3. **Packet construction:** create a link-state packet with router ID, neighbours, costs, sequence number and age.
4. **Flooding:** distribute new packets to routers throughout the routing domain. Reject duplicates/obsolete advertisements using sequence information; age out stale data.
5. **Shortest-path calculation:** each router builds the graph and runs Dijkstra with itself as root, obtaining a shortest-path tree and next hops.
6. **Example and evaluation:** with A-B=2, B-C=1 and A-C=7, A chooses C via B at cost 3. Link state supports a detailed topology view and fast route recomputation, but needs memory, CPU and control-message overhead. OSPF uses this approach.

Remember: One sentence definition + diagram + five stages + example/advantage is a strong 5-mark structure.

Source: Tanenbaum Fig.5-12, printed p.375 (PDF p.399), and section 5.2.5.

PART B - UNIT 4

Services provided by TCP

Understand it first

IP sends packets between network interfaces. An application often needs more: "Give the other program these bytes, in order, without duplicates." TCP provides a reliable, ordered byte-stream service between application endpoints.

A **byte stream** is a sequence of bytes, not a preserved sequence of application messages. Sending two writes does not guarantee the receiver will obtain exactly two reads. The application must define any message boundaries it needs.

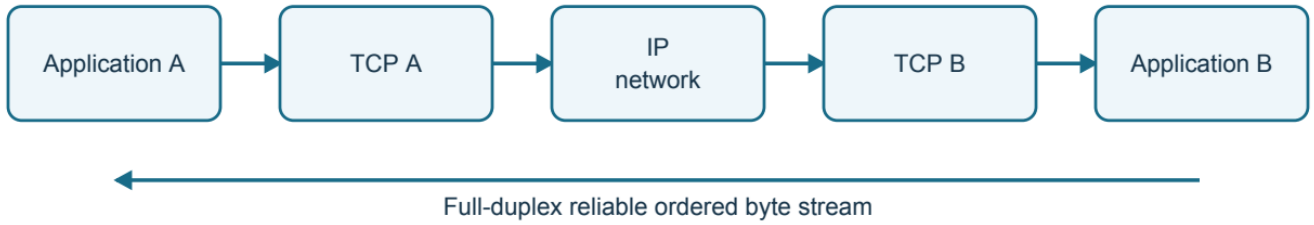


Diagram to draw: Draw Application A ↔ TCP A ↔ IP network ↔ TCP B ↔ Application B. Show arrows both ways and label ordered byte stream.

5-mark answer

1. **Process-to-process delivery:** TCP ports identify application endpoints; IP addresses plus ports identify the communicating endpoints.
2. **Connection-oriented service:** TCP establishes state through a three-way handshake before ordinary data transfer.
3. **Reliable, ordered byte stream:** sequence numbers, checksums, acknowledgements and retransmissions detect/manage loss, corruption and duplicates; the receiving application gets bytes in order while the connection works.
4. **Full-duplex transfer:** both ends can send and receive data independently over the connection.
5. **Flow control:** the receiver advertises available receive capacity through its window, helping prevent receiver overflow.
6. **Congestion control:** the sender adjusts outstanding data/rate to network conditions. This protects the network, whereas flow control protects the receiver. TCP can report connection failure; it cannot promise success through a permanent outage.

Worked example / explanation: If bytes 100-149 arrive but 150-199 are missing, TCP cannot expose later data as if the missing bytes never existed. It tracks order and requests/retransmits what is needed.

Remember: Reliable + ordered + byte stream + connection-oriented + full duplex + flow/congestion control.

Source: U4 pp.23,25-31; Tanenbaum sections 6.5.1-6.5.3, printed pp.552-556 (PDF pp.576-580).

PART B - UNIT 4

TCP connection management

Understand it first

Both sides must agree to communicate and learn each other's initial sequence numbers. A **SYN** synchronizes sequence numbers. An **ACK** indicates the next sequence number expected from the other side.

Think of the handshake as: "I want to start at x." "I heard x, and I start at y." "I heard y." Three messages confirm communication in both directions. SYN consumes one position in the sequence-number space even when it carries no application data.

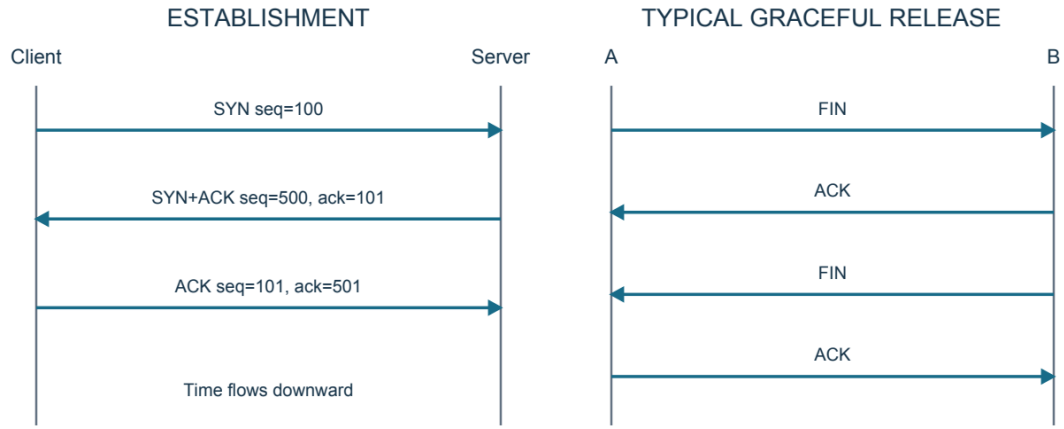


Diagram to draw: Draw the three labelled arrows for establishment. If the question says connection management, add the four-arrow FIN/ACK release diagram. Time flows downward.

5-mark answer

1. **1. SYN:** client sends SYN=1 and sequence x; it enters SYN-SENT while the server was listening.
2. **2. SYN+ACK:** server replies with its sequence y and acknowledgement x+1; it enters SYN-RECEIVED.
3. **3. ACK:** client acknowledges y+1, normally using sequence x+1. The server enters ESTABLISHED when it receives this valid ACK; ordinary data exchange can proceed.
4. **Graceful release:** each direction closes independently. A typical exchange is FIN → ACK → FIN → ACK; the two middle actions may sometimes be combined.
5. **FIN versus RST:** FIN is an orderly end of one sending direction. RST aborts/resets a connection. TIME-WAIT on the endpoint completing active close allows handling of delayed segments and retransmitted FINs.

Worked example / explanation: Practice values: client SYN seq=100; server SYN+ACK seq=500, ack=101; client ACK seq=101, ack=501. SYN consumes one sequence number; an ACK alone consumes none.

Remember: ACK = next expected byte/sequence number. For data: seq=101 and 50 bytes means next expected = 151, not 102.

Source: U4 pp.27-29; Tanenbaum Fig.6-37 and section 6.5.6, printed pp.560-564 (PDF pp.584-588).

PART B - UNIT 4

TCP header format

Understand it first

The TCP header carries the information needed for connection management, byte ordering, acknowledgements, error detection and flow control. It is **20 bytes minimum** and can grow to **60 bytes** with options.

Draw a 32-bit-wide rectangle. The mandatory portion has five rows. Learn those rows in order rather than memorising every field as an unrelated fact.

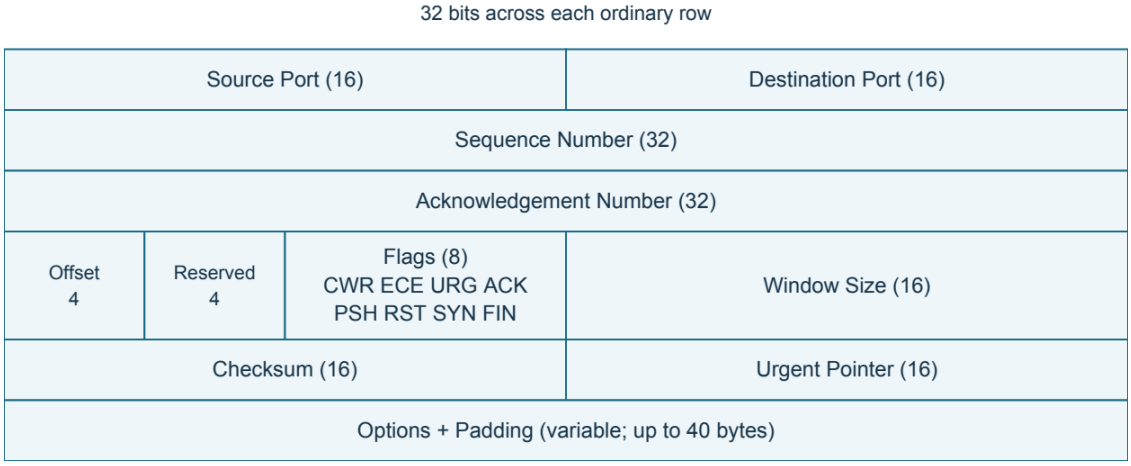


Diagram to draw: Use this redraw or the original Tanenbaum figure in the appendix. The original blank shaded field is Reserved (4). Do not combine 4 reserved bits with only 6 flags: that leaves 2 bits missing.

5-mark answer

1. **Ports:** source and destination ports are 16 bits each and identify the application endpoints.
2. **Sequence number (32):** normally the number of the first data byte in the segment. **Acknowledgement number (32):** next in-order byte expected when ACK is set.
3. **Data offset/header length (4):** TCP header size in 32-bit words. The textbook layout then has 4 reserved bits and 8 one-bit flags.
4. **Flags:** SYN establishes; FIN closes one direction; RST resets; ACK validates the acknowledgement; PSH requests prompt delivery; URG marks the urgent pointer as meaningful; ECE/CWR support explicit congestion notification.
5. **Window (16):** receiver-advertised flow-control capacity. **Checksum (16):** detects errors over the TCP header, data and an IP pseudoheader.
6. **Urgent pointer (16)** is used with URG. Options and padding may follow the fixed header; padding preserves a 32-bit boundary.

Worked example / explanation: Fourth row: 4 (offset) + 4 (reserved) + 8 (flags) + 16 (window) = 32 bits. The older classic six-flag drawing uses 6 reserved bits instead; do not mix the two layouts.

Remember: Ports → sequence → acknowledgement → length/flags/window → checksum/urgent → options.

Source: Tanenbaum Fig.6-36, printed pp.557-559 (PDF pp.581-583); RFC 9293, section 3.1.

PART B - UNIT 4

UDP header format

Understand it first

UDP gives applications a small, connectionless datagram service. It adds port numbers so incoming data can reach the appropriate application endpoint, but does not establish a connection or provide built-in retransmission and ordering.

Unlike TCP's byte stream, UDP preserves datagram boundaries. UDP itself does not guarantee that a datagram arrives, arrives once, or arrives in order. An application can add reliability when it needs it, as TFTP does.

32 bits across each ordinary row

Source Port (16)	Destination Port (16)
UDP Length (16)	Checksum (16)

Diagram to draw: Copy two rows: Source port (16) | Destination port (16); Length (16) | Checksum (16).

5-mark answer

1. The UDP header has a fixed size of **8 bytes**: four 16-bit fields arranged in two 32-bit rows.
2. **Source port:** identifies the sending endpoint and can be used for replies.
3. **Destination port:** identifies the receiving application endpoint.
4. **Length:** counts the entire UDP datagram, including the 8-byte header and its data.
5. **Checksum:** detects errors over header, data and an IP pseudoheader. It is optional for ordinary UDP over IPv4 and normally mandatory over IPv6; it does not repair data.
6. UDP has low header overhead and no built-in handshake, retransmissions, flow control or congestion control. Examples include conventional DNS queries, DHCP and TFTP.

Worked example / explanation: If application data is 12 bytes, UDP Length = 8 + 12 = 20 bytes. The enclosing IP header is not included in this UDP length.

Remember: UDP is unreliable in delivery, but it still has checksum-based error detection. Do not write "UDP has no error checking".

Source: U4 pp.32-34; Tanenbaum Fig.6-27, printed p.542 (PDF p.566); IPv6 checksum rule: RFC 8200 section 8.1.

PART B - UNIT 5

DNS: name space, records and resolution

Understand it first

Humans prefer names such as www.example.com; IP forwarding uses addresses. DNS is a **distributed, hierarchical naming system** that looks up records associated with domain names, including their IP addresses.

The hierarchy is read from right to left: root (.), top-level domain (.com), registered domain (example.com), then a host/subdomain (www). A **zone** is an administratively managed part of the name space. Its primary server holds the maintained data; secondary servers obtain copies and can also answer authoritatively.

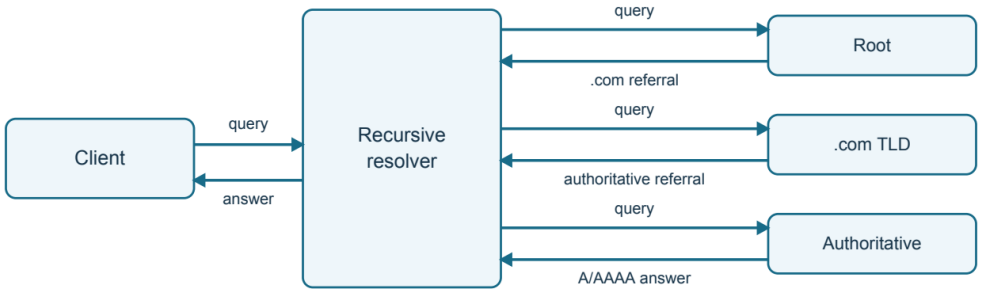


Diagram to draw: Draw the domain hierarchy and the query flow. The client asks the resolver; the resolver separately queries root, TLD and authoritative servers, receiving replies at each stage.

5-mark answer

1. A host's resolver sends a query to a recursive DNS resolver, often supplied by the network or configured by the user.
2. If a valid cached answer exists, the resolver can return it immediately. Otherwise, it queries the hierarchy.
3. The root refers it to the appropriate top-level-domain servers; a TLD server refers it to authoritative servers for the requested domain.
4. The authoritative server supplies the requested record, and the resolver returns the answer to the client. Responses can be cached for their TTL.
5. A resource record contains name, TTL, class, type and value. Important types are A (IPv4), AAAA (IPv6), MX (mail exchanger), NS (name server), CNAME (alias), and PTR (reverse lookup).
6. Conventional DNS uses port 53 over UDP or TCP. Root servers generally provide referrals; they do not store every host's address. DNS resolution finds an address; it does not itself download the webpage.

Worked example / explanation: For www.example.com: root → referral for .com; .com → referral for example.com; example.com authoritative server → A/AAAA record. This is a teaching sequence, not a live lookup.

Remember: Hierarchy = how names are organised. Resolution = how an answer is obtained. Cache = how repeated work is avoided.

Source: U5 pp.8-18; Tanenbaum Figs.7-1,7-6 and sections 7.1.1-7.1.3, printed pp.612-621 (PDF pp.636-645).

PART B - UNIT 5

Email architecture, components and agents

Understand it first

Email is usually **store-and-forward**: servers accept and store messages, then pass them onward. The sender and receiver do not need to be online at the same instant. A user agent is the program a person uses to compose and read mail; mail servers move and store it.

Do not confuse sending with reading. SMTP transfers outgoing messages. POP3 or IMAP lets the receiver's mail client access stored messages. Webmail may use HTTPS between the browser and its provider.



Diagram to draw: Draw sender UA → sending MTA → receiving MTA/MDA → mailbox → receiver UA. Label SMTP on sending/transfer arrows and POP3/IMAP on mailbox access.

5-mark answer

1. **Mail User Agent (MUA/UA):** supports composition, addressing, reading, replying and organising messages.
2. **Mail Transfer Agent (MTA):** transfers mail between servers using SMTP; it can queue messages and retry delivery.
3. **Mail Delivery Agent (MDA):** performs local delivery into the recipient's mailbox. A mailbox stores received mail; an outgoing queue holds mail waiting for transfer.
4. **Transfer path:** sender UA submits mail → sending mail server → receiving mail server → mailbox → receiver UA using POP3/IMAP.
5. **Message structure:** a message has header fields (From, To, Subject, Date), a blank line and a body. The transport envelope supplies delivery information and is distinct from the message header.
6. **Services:** composition, transfer, delivery/status reporting, display and message disposition (save/delete/reply). MIME adds content types and encodings for attachments and non-ASCII content; it is not a separate transport connection.

Remember: UA = user-facing program. MTA = transfer. MDA = local delivery. Mailbox = storage. SMTP sends; POP3/IMAP access.

Source: U5 pp.22-37; Tanenbaum Fig.7-7 and sections 7.2.1-7.2.3, printed pp.624-630 (PDF pp.648-654).

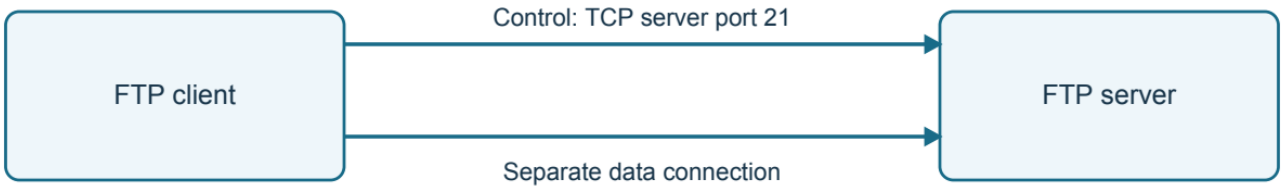
PART B - UNIT 5

FTP: File Transfer Protocol

Understand it first

FTP lets a client upload or download files from a server. It separates the conversation about the transfer from the actual file bytes: one TCP connection carries commands and replies, while a separate TCP connection carries data.

A directory listing is also data, so it can use the data connection. The control connection can remain open while multiple transfers take place.



Active: server initiates data. Passive: client initiates data.

Diagram to draw: Draw FTP Client and FTP Server with two separate labelled connections: control TCP 21 and data TCP connection. Add active/passive initiation arrows if asked.

5-mark answer

1. FTP is an application-layer client-server protocol for file transfer over TCP.
2. **Control connection:** the client connects to server TCP port 21 for login, commands and responses.
3. **Data connection:** a separate TCP connection carries a file or directory listing. Commands include USER/PASS for login, RETR for download, STOR for upload and QUIT to finish.
4. **Active mode:** the server initiates the data connection to a client-selected endpoint; the traditional server data port is 20.
5. **Passive mode:** the server announces a listening data endpoint and the client initiates the data connection. The server data port is not always 20.
6. TCP supplies reliable ordered transfer. Basic FTP does not encrypt credentials or data; secure variants/other protocols are needed where confidentiality is required.

Remember: Two connections. Control = 21. Data = separate; port 20 is the classic active-mode server port, not a universal data port.

Source: U5 pp.38-39,45-46.

PART B - UNIT 5

TFTP: Trivial File Transfer Protocol

Understand it first

TFTP is a small file-transfer protocol used where implementation simplicity matters, such as network booting or moving device configuration files. It uses UDP, but its own block numbers, acknowledgements and timeouts provide a basic transfer mechanism.

This is an important distinction: UDP does not provide retransmission, but an application protocol using UDP can implement it. TFTP is not simply FTP with a different port.

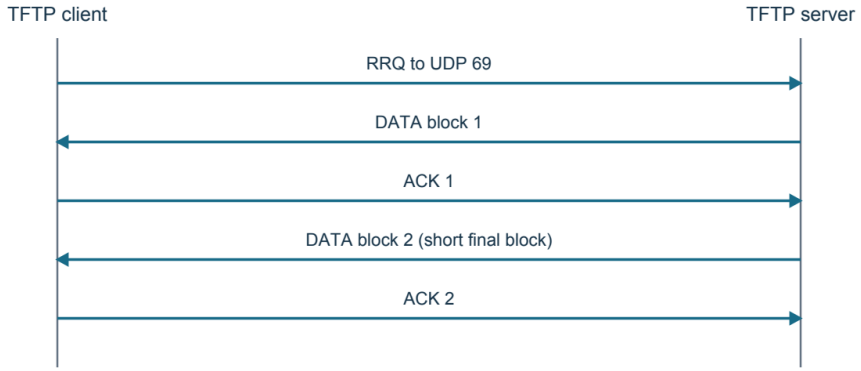


Diagram to draw: Draw RRQ ->, DATA block 1 <-, ACK 1 ->, DATA block 2 <-, ACK 2 → between client and server. Mark the final short DATA block.

5-mark answer

1. TFTP is a simple application-layer file-transfer protocol carried over UDP. It supports reading and writing files without FTP-style login or directory-listing features.
2. The client sends an RRQ (read request) or WRQ (write request) to the server's UDP port 69. The transfer then uses selected transfer identifiers/UDP ports.
3. In basic TFTP, a sender transmits a numbered DATA block, normally containing up to 512 bytes, and waits for its ACK before sending the next block.
4. If the expected response does not arrive, a timeout causes retransmission. This is stop-and-wait operation.
5. A DATA block shorter than 512 bytes marks the end in the base protocol. A file exactly divisible by 512 needs a final zero-byte DATA block.
6. Its small size suits boot/configuration tasks, but basic TFTP provides neither user authentication nor encryption. Extensions may negotiate different block sizes; the 512-byte rule here describes the base protocol.

Remember: UDP 69 starts the request. DATA/ACK give TFTP its own reliability. Basic block size = 512 bytes.

Source: U5 p.40; RFC 1350, sections 2-5.

PART B - UNIT 5

Telnet: remote terminal access

Understand it first

Telnet lets a user type commands on a local computer while the commands run on a remote computer. The remote output comes back to the local screen. It is a remote terminal protocol, not primarily a file-transfer protocol.

Different systems can have different terminal conventions. Telnet uses a **Network Virtual Terminal (NVT)** representation so a client and server can translate between their local formats and a common network form.

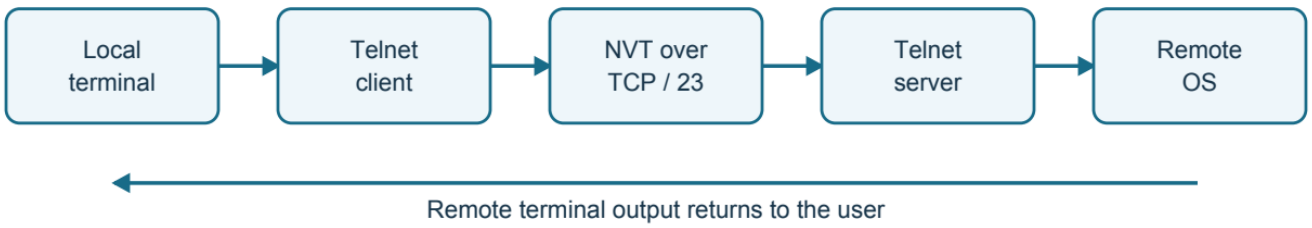


Diagram to draw: Draw local terminal → Telnet client → TCP/IP network → Telnet server → remote OS. Label NVT representation across the network and return output.

5-mark answer

1. Telnet is an application-layer protocol that provides bidirectional text-based remote terminal access over TCP, conventionally on server port 23.
2. The local Telnet client establishes a TCP connection with a Telnet server on the remote machine.
3. The client converts local terminal input to the NVT representation and sends it to the server.
4. The server translates the input for its system, executes authorised commands through the login/session environment, and sends terminal output back.
5. Telnet supports option negotiation (WILL, WON'T, DO, DON'T) and control operations such as interrupt process and erase character.
6. Basic Telnet sends data, including ordinary login credentials, without encryption. SSH is preferred for secure remote login.

Worked example / explanation: Typing a directory-listing command through Telnet operates on the remote system, not your local laptop.

Remember: Telnet = remote login, TCP 23, NVT, plaintext. FTP/TFTP = file transfer.

Source: U5 pp.42-43,50-55; RFC 854 for NVT and option negotiation.

PART B - UNIT 5

POP3: accessing stored email

Understand it first

POP3 allows a mail client to retrieve messages from a server mailbox. It is designed around a simple download-oriented model, so a user commonly reads the retrieved mail locally.

Downloading does not automatically mean every server copy is deleted. Deletion is controlled by the client/protocol session. Avoid the inaccurate statement that POP3 always deletes downloaded email.

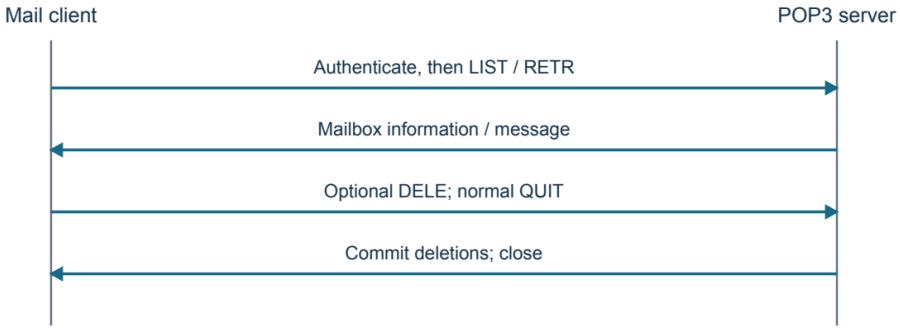


Diagram to draw: Draw client → POP3 server: login/LIST/RETR; server → client: messages. Show an optional DELE/QUIT step, not unconditional deletion.

5-mark answer

1. Post Office Protocol version 3 is an application-layer mail-access protocol using TCP, conventionally port 110.
2. **Authorization:** the client connects and authenticates to the server.
3. **Transaction:** the client can inspect the mailbox and retrieve messages. Typical commands include STAT, LIST and RETR.
4. **Deletion:** DELE marks a message for deletion; RSET can clear deletion marks during the transaction.
5. **Update:** on a normal QUIT from the transaction state, the server commits marked deletions and ends the session.
6. POP3 is simple and suitable for downloading mail, but offers less server-side folder management and multi-device synchronisation than IMAP. SMTP is still used to send outgoing mail.

Remember: POP3 retrieves; SMTP sends. Remember Authorization → Transaction → Update.

Source: U5 pp.22,37; Tanenbaum printed p.644 (PDF p.668); RFC 1939, sections 3-6.

PART B - UNIT 5

SMTP: sending and transferring email

Understand it first

SMTP is the protocol mail systems use to submit and relay outgoing messages. It is a **push** protocol: the sending side opens a connection and transfers mail to the receiving server. It does not provide mailbox browsing or inbox synchronisation.

An MTA can keep a message in its queue and retry when the receiving server is temporarily unavailable. Once delivered to the receiving system, POP3/IMAP can provide the recipient's mailbox access.

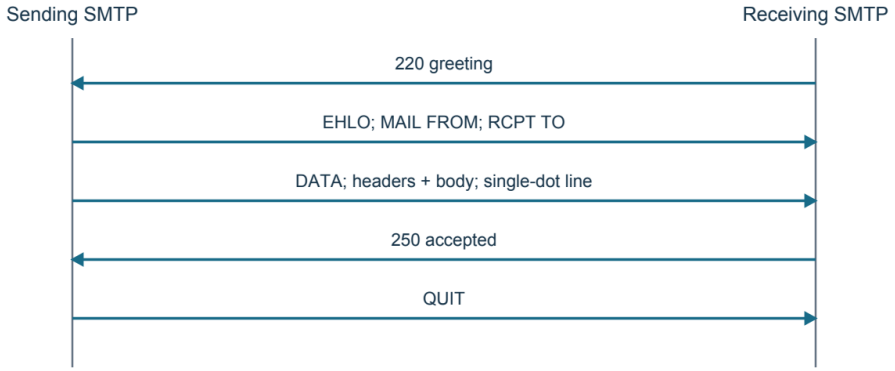


Diagram to draw: Draw SMTP client and server with greeting, EHLO, MAIL FROM, RCPT TO, DATA, acceptance and QUIT. Or use the email architecture with SMTP labelled between MTAs.

5-mark answer

1. Simple Mail Transfer Protocol is an application-layer protocol using TCP. Server-to-server relay conventionally uses port 25; client submission commonly uses port 587.
2. The SMTP client connects; the server sends a greeting, and the client identifies itself with HELO or EHLO.
3. MAIL FROM identifies the envelope sender and RCPT TO identifies a recipient. Repeat RCPT TO for additional recipients.
4. DATA begins transmission of the message headers, blank line and body. A line containing only a dot terminates the data in the basic exchange.
5. The server returns status replies, and QUIT ends the session. SMTP supports store-and-forward transfer through mail servers.
6. MIME supplies a way to represent attachments/content types. SMTP transfers messages; it does not replace POP3/IMAP for accessing stored mail.

Remember: SMTP = Simple Mail Transfer Protocol, not "Transport Protocol". It is an application-layer protocol carried by TCP.

Source: U5 pp.23-35; Tanenbaum section 7.2.4, printed pp.638-642 (PDF pp.662-666); RFC 6409 for submission port.

PART B - UNIT 5

IMAP: server-based email access

Understand it first

IMAP lets a client view and manage mail kept on a server. This makes it suitable for using the same mailbox from a phone and a laptop: both can see server-side folders and message flags such as read/unread.

A client can cache messages for offline use, so "IMAP never downloads mail" is false. The distinction is that the server remains the main shared mailbox, with richer remote management.

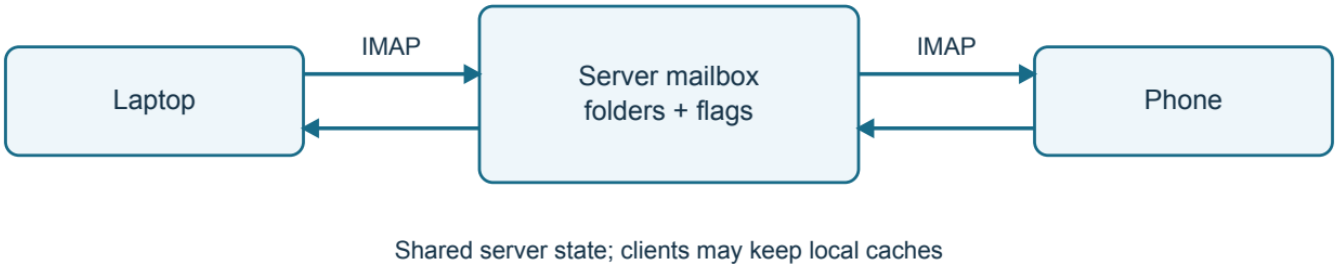


Diagram to draw: Draw one server mailbox connected bidirectionally to a laptop and phone. Label folders/flags stored on server and IMAP synchronisation on both connections.

5-mark answer

1. Internet Message Access Protocol is an application-layer mail-access protocol carried over TCP, conventionally port 143 (993 for implicit TLS).
2. The client connects, authenticates, and selects a mailbox/folder to access.
3. It can list messages, fetch headers or bodies, search on the server, and retrieve only needed portions.
4. It manages server-side folders and flags such as seen/unseen and deleted, allowing clients to share a consistent mailbox view.
5. Messages normally remain on the server until the user deletes them; clients may keep local caches and synchronise changes.
6. Compared with POP3, IMAP gives richer mailbox management and works better for multiple devices, but is more complex. SMTP remains the protocol used for outgoing mail.

Remember: IMAP accesses and synchronises mail. ICMP reports IP errors. The similar-looking abbreviations describe completely different jobs.

Source: U5 p.37; Tanenbaum section 7.2.5, printed pp.644-645 (PDF pp.668-669).

FINAL REVISION

Keep these distinctions clear

Dijkstra, distance vector and link state

Point	Dijkstra	Distance vector	Link state
What it is	Shortest-path calculation	Routing method	Routing method
Knowledge used	Weighted graph	Neighbours' distance vectors	Topology database
Main rule	Choose nearest; relax	Direct link + neighbour distance	Advertise links; run Dijkstra
Your lab	Q8	Q10	Q11

The three email protocols

Feature	SMTP	POP3	IMAP
Main job	Send / relay mail	Retrieve mail	Access / manage mail
Common TCP port	25 relay; 587 submission	110	143
Typical model	Push to server	Download oriented	Server-based mailbox
Folders / flags	Not mailbox access	Limited management	Rich server management

Address-related protocols

Protocol	Question it answers
DNS	What records/IP addresses belong to this name?
ARP	What is the MAC for this local IPv4 next hop?
RARP	Which IP was assigned to my known MAC?
DHCP	What IP, mask, gateway and DNS settings should I use?
ICMP	What IP problem occurred / does this host respond?

Common ports are conventional base service ports. Encrypted service variants may use other ports; IMAP with implicit TLS uses 993.

FINAL REVISION

Recall and diagram checklist

Cover the answers and explain these aloud

Why is the next hop for A to C equal to B?
Because A-B-C costs 3, less than direct A-C cost 7. B is the first step, not the final destination.

Why do Q8 and Q11 share code?
Link state supplies a graph; Dijkstra calculates shortest paths from that graph.

What exactly does the subnet mask do?
It marks network bits with 1s and host bits with 0s. IP AND mask yields the network address.

What happens when the bucket cannot hold a packet?
Your lab drops that whole arriving packet, then transmits up to the output rate.

Why does Caesar use %26?
There are 26 uppercase letters; modulo wraps the shifted value around the alphabet.

How do IPv4 and IPv6 header lengths differ?
IPv4: 20-60 bytes. IPv6: fixed 40-byte base header plus optional extension headers.

TCP sequence 101 carries 50 bytes. What ACK is expected?
151, assuming those bytes and all preceding bytes are received in order.

Does UDP detect errors?
Yes, its checksum detects errors. UDP does not provide built-in retransmission.

Does SMTP read your inbox?
No. POP3/IMAP access stored mail; SMTP sends/relays it.

Does POP3 always delete downloaded messages?
No. Deletion depends on commands/settings and session completion.

Practise these drawings from memory

Three-router weighted graph and distance table; A/B/C address split; leaky bucket; encryption chain; IPv4/IPv6 headers; ARP request/reply; RARP; DHCP DORA; link-state stages; TCP handshake and release; TCP/UDP headers; DNS hierarchy/resolution; email architecture; FTP connections; TFTP DATA/ACK; Telnet/NVT; POP3 exchange; IMAP multi-device mailbox.

A good diagram needs labels, arrow directions and field sizes where relevant. A decorative box without those details is not enough.

DIAGRAM SOURCE APPENDIX

Original textbook headers: IP

These are figure crops from your local Tanenbaum 5th edition PDF. Use the simplified teaching diagrams when you need an easier version to reproduce.

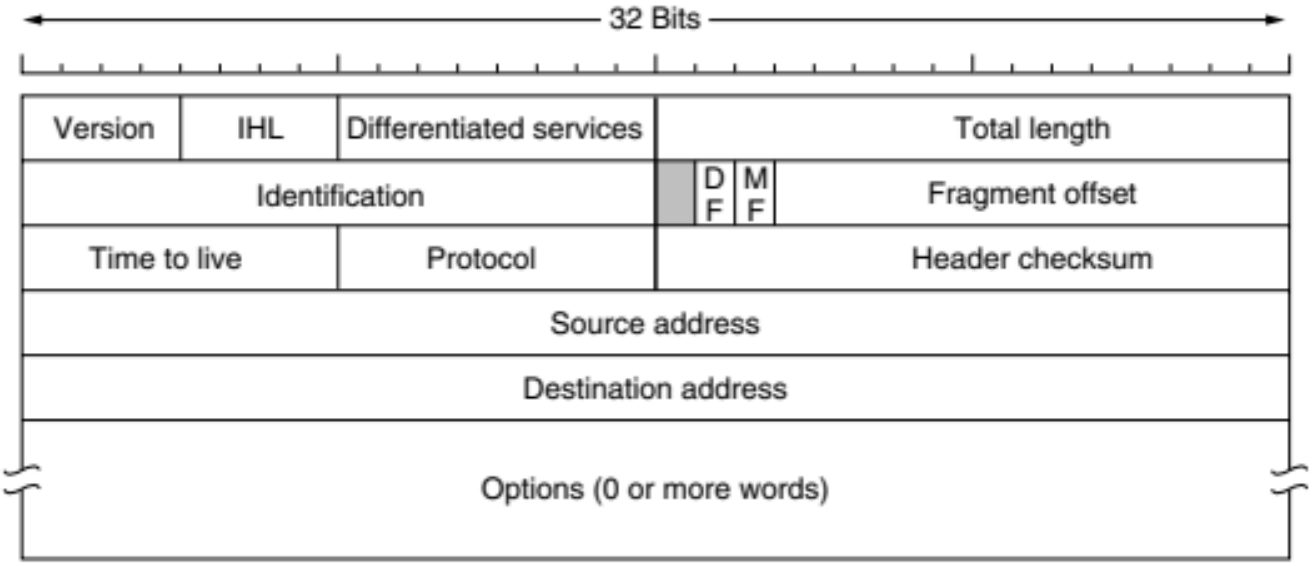


Figure 5-46. The IPv4 (Internet Protocol) header.

Fig.5-46: IPv4 header. Printed p.439; PDF p.463.

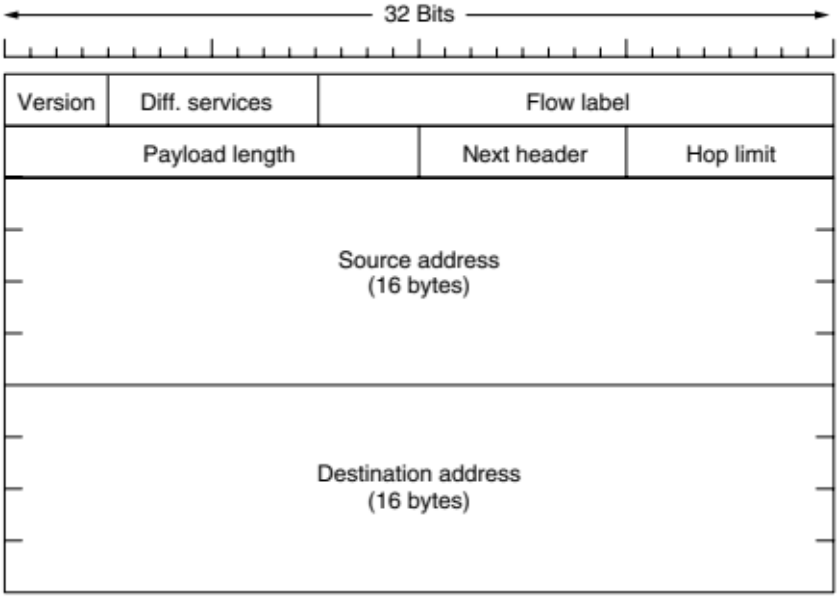


Figure 5-56. The IPv6 fixed header (required).

Fig.5-56: IPv6 fixed header. Printed p.458; PDF p.482.

DIAGRAM SOURCE APPENDIX

Original textbook headers: transport

These are figure crops from your local Tanenbaum 5th edition PDF. Use the simplified teaching diagrams when you need an easier version to reproduce.

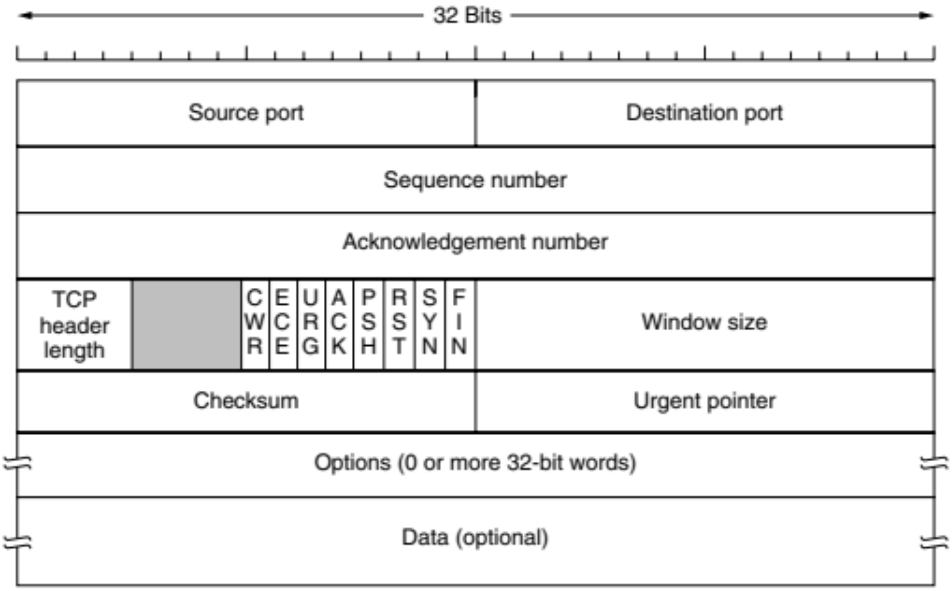


Figure 6-36. The TCP header.

Fig.6-36: TCP header. Printed p.557; PDF p.581.

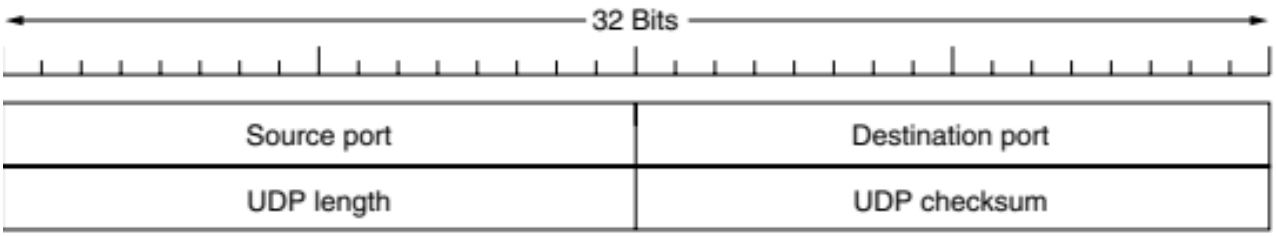


Figure 6-27. The UDP header.

Fig.6-27: UDP header. Printed p.542; PDF p.566.

DIAGRAM SOURCE APPENDIX

Original textbook routing and shaping

These are figure crops from your local Tanenbaum 5th edition PDF. Use the simplified teaching diagrams when you need an easier version to reproduce.

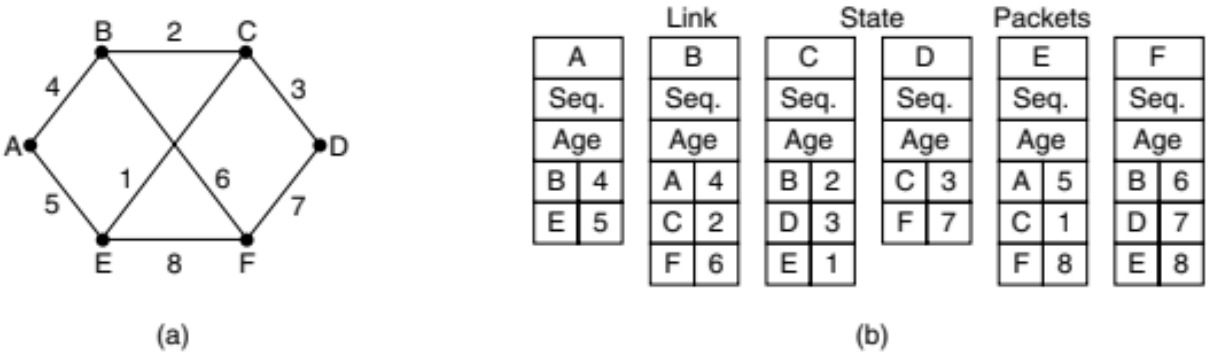


Figure 5-12. (a) A network. (b) The link state packets for this network.

Fig.5-12: network and link-state packets. Printed p.375; PDF p.399.

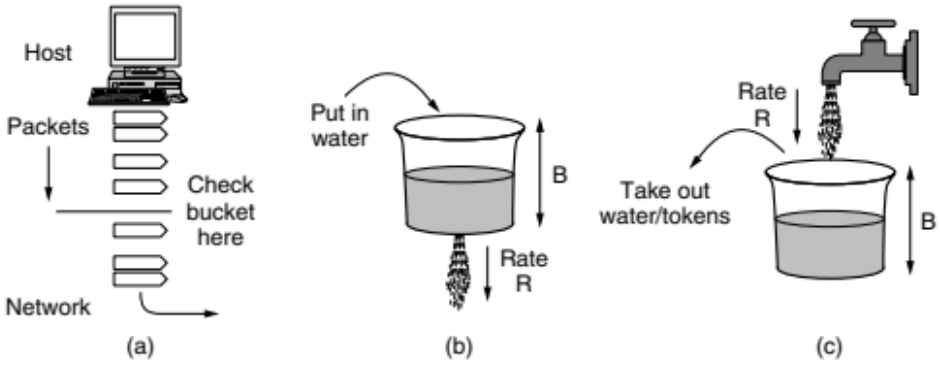


Figure 5-28. (a) Shaping packets. (b) A leaky bucket. (c) A token bucket.

Fig.5-28: shaping, leaky bucket (b), token bucket (c). Printed p.408; PDF p.432.

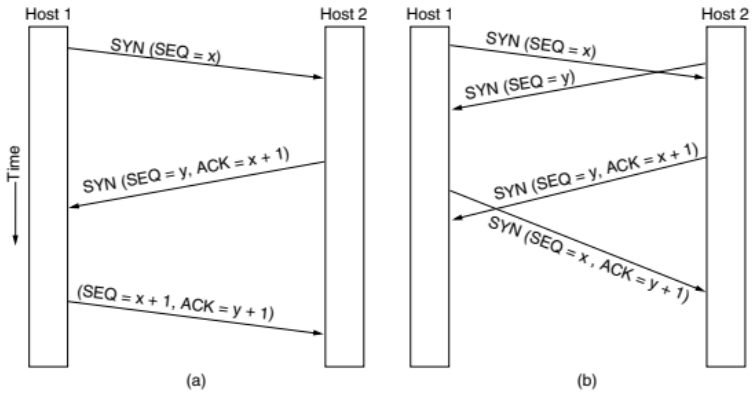


Figure 6-37. (a) TCP connection establishment in the normal case. (b) Simultaneous connection establishment on both sides.

Fig.6-37: normal TCP establishment (a) and simultaneous open (b). Printed p.561; PDF p.585.

DIAGRAM SOURCE APPENDIX

Original textbook application diagrams

These are figure crops from your local Tanenbaum 5th edition PDF. Use the simplified teaching diagrams when you need an easier version to reproduce.

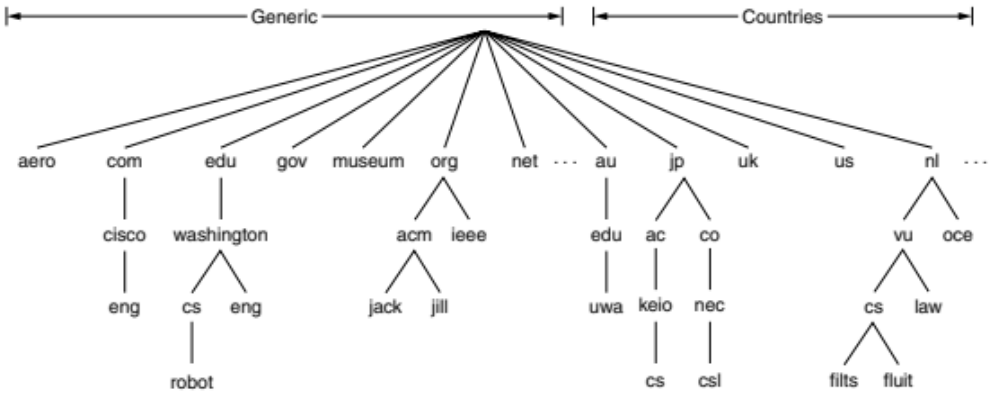


Figure 7-1. A portion of the Internet domain name space.

Fig.7-1: domain name hierarchy. Printed p.613; PDF p.637.

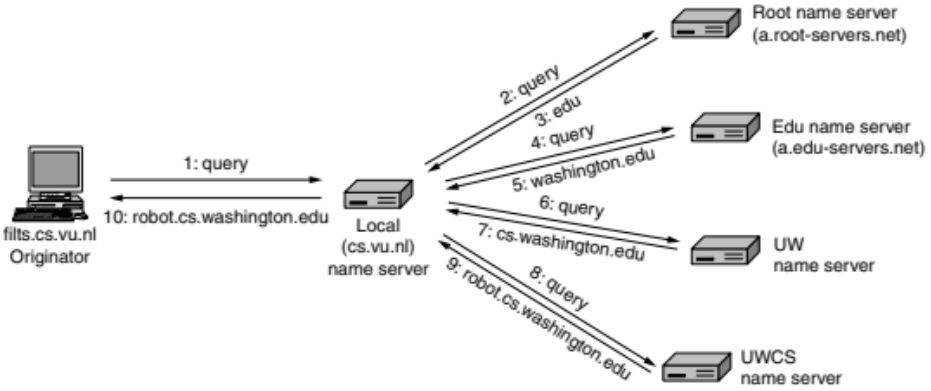


Figure 7-6. Example of a resolver looking up a remote name in 10 steps.

Fig.7-6: DNS resolution. Printed p.620; PDF p.644.

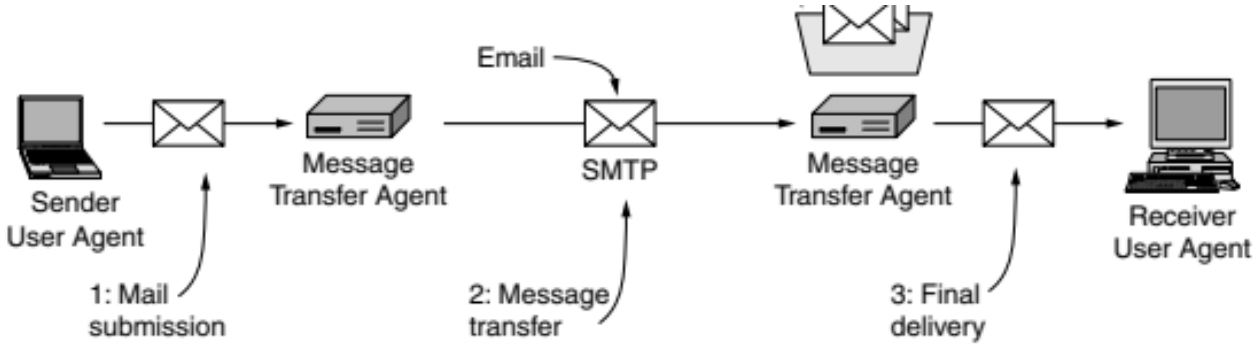


Figure 7-7. Architecture of the email system.

Fig.7-7: email architecture. Printed p.624; PDF p.648.

SOURCES AND CORRECTIONS

Where the explanations came from

Local source key

U3: sir's Unit 3 notes
/Users/timothysaxena/Downloads/MGIT/Sem-V/2. CN/cn notes sir/UNIT 3.pdf

U4: sir's Unit 4 notes
/Users/timothysaxena/Downloads/MGIT/Sem-V/2. CN/cn notes sir/UNIT 4.pdf

U5: sir's Unit 5 notes
/Users/timothysaxena/Downloads/MGIT/Sem-V/2. CN/cn notes sir/UNIT 5.pdf

Tanenbaum and Wetherall, Computer Networks, 5th edition
/Users/timothysaxena/Downloads/MGIT/Sem-V/2. CN/TB/computer-networks-tanenbaum-5th-edition.pdf

U3/U4/U5 page numbers refer to PDF file pages. Textbook citations give printed and PDF page numbers where a figure/section was directly checked. In this file, the cited textbook PDF page is the printed page plus 24.

Corrections applied to course notes

- IPv6 uses Traffic Class 8 bits and Flow Label 20 bits; the 4/24 split in U3 p.46 is outdated.
- TCP diagrams must account for all bits in each row. This guide uses the textbook's 4 reserved bits and 8 flags.
- A TCP acknowledgement is the next expected in-order byte. RST resets; FIN closes one sending direction.
- UDP has checksum-based error detection. The checksum is optional in IPv4 but normally required in IPv6.
- SMTP means Simple Mail Transfer Protocol; IMAP means Internet Message Access Protocol. POP3 uses TCP, not UDP.
- FTP data is not always on server port 20; passive mode uses a negotiated server data port.
- Basic TFTP starts at UDP 69 and then uses transfer ports. POP3 does not automatically delete every retrieved message.

Official supplements used for precise details

IPv6 header and checksum - https://www.rfc-editor.org/rfc/rfc8200.html

TCP header - https://www.rfc-editor.org/rfc/rfc9293.html

Basic TFTP - https://www.rfc-editor.org/rfc/rfc1350.html

POP3 states and deletion - https://www.rfc-editor.org/rfc/rfc1939.html

Telnet NVT and negotiation - https://www.rfc-editor.org/rfc/rfc854.html

Email submission - https://www.rfc-editor.org/rfc/rfc6409.html

TLS mail access ports - https://www.rfc-editor.org/rfc/rfc8314.html